

Virtual chiplets for on-device AI

Team guide: what LOOM is, how every part works, and what to tell the judges

iQOO Hackathon 2026 · Bengaluru · built and measured on the iQOO 15

All numbers in this guide were measured on our iQOO 15 on 10 October 2026, unless marked as simulated.

CONTENTS

What is in this guide

#	Section	Covers
1	LOOM on one page	the pitch and the numbers
2	The problem	why camera AI overheats phones
3	The idea	virtual chiplets and the Weaver
4	The five chiplets	what each one does and its levels
5	Why these models and settings	every choice and its reason
6	How the Weaver decides	the rules, in order
7	Two techniques that make it work	hot-swap and slowing the camera sensor
8	The quantum planner	QUBO, QAOA, and how we check it
9	The phone app	what the user sees and hears
10	The dashboard, panel by panel	what every number means
11	Measured results	15-minute test on the iQOO 15
12	How LOOM helps iQOO	
13	Demo script for the judges	what to do, what they will see
14	Questions judges may ask	with honest answers
15	Honest limits and next steps	
16	Appendix	files, scripts and a glossary

How to use this guide

Read sections 1, 11, 13 and 14 the night before: they are what you will say and show. Sections 3 to 10 are there so that every team member can answer a deep question about any part.

1 · LOOM ON ONE PAGE

Same phone, same job, much cooler

Camera AI on a phone gets hot and drains the battery. Judges have already criticised another team for exactly this with their NPU app. **LOOM fixes it automatically.** We split the camera AI into five small blocks, **virtual chiplets**, and a controller called the **Weaver** retunes them while the app runs: a smaller model, a slower camera sensor, a dimmer screen, speaking less often. Nobody touches the phone.

-44%	Flat	0.01 ms	15 / 15
battery draw vs an ordinary app (586 vs 1,048 mA)	heat for 15 minutes (0.84 → 0.85)	gap when the AI model is swapped live	quantum answers match brute force

What it is, in four lines

- **A blind-assist camera app** (our demo vehicle): it detects people, chairs, cars and other obstacles and speaks short, stable sentences such as “Person ahead, chair on the left”.
- **A runtime underneath it** that treats each stage of the AI pipeline as a tunable block with quality levels.
- **A control loop (the Weaver)** that checks heat, speed and battery every second and changes one block at a time, always with a written reason.
- **A quantum planner** that decides ahead of time which settings are best for every heat level, solved with QAOA in Qiskit and checked against all possible answers.

The 30-second pitch

> **Say this**
> “Camera AI on a phone heats it up and drains the battery. LOOM fixes that automatically. We split the camera AI into five virtual chiplets, and a controller called the Weaver retunes them while the app runs. On a real iQOO 15, LOOM drew 44% less battery than an ordinary app doing the same job and held the heat flat instead of letting it climb. A QAOA optimiser in Qiskit plans the best settings for every heat level, and every answer is checked by brute force. Nobody touches the phone.”

The three things judges must remember

1. **It is automatic.** The phone only shows the camera. Every decision appears on the laptop with its reason.
2. **It is measured on the real phone.** 15-minute runs on the iQOO 15, logged every second.
3. **It is honest.** The quantum part runs on a simulator and is verified; we say what is measured and what is not.

2 · THE PROBLEM

Why camera AI overheats a phone

A camera AI app does several heavy things at once, 30 times a second: the camera sensor and image processor capture frames, the GPU runs a neural network on each one, the screen shows a bright preview, and the speaker talks. An ordinary app runs all of this at full quality all the time, whether it is needed or not.

What happens next, as we measured on the iQOO 15 with an ordinary app (everything at maximum):

1. The phone draws about **1,050 mA** from the battery.
2. Its thermal headroom (Android's own heat measure) climbs from 0.61 to **0.98** in 15 minutes and is still rising.
3. Android reports a **MODERATE** thermal warning and starts slowing the phone down: the app falls from 18.7 to **12 frames per second**.
4. So the user gets a hot phone, a draining battery, *and* a slower app anyway.

Why the usual fixes are not enough

- **Better cooling hardware** moves heat out, but it cannot stop the software from making heat it does not need.
- **Always using the smallest model** wastes quality when the phone is cool and does not need saving.
- **Android's own throttling** acts late and bluntly: it slows everything, including what matters most.

> **The gap LOOM fills**
> Nobody decides, moment by moment, *which* part of the AI work is worth its heat right now. LOOM does exactly that, and gives up the least important quality first.

3 · THE IDEA

Virtual chiplets and the Weaver

A **chiplet** in hardware is a small, separate block of a processor that can be tuned or swapped on its own. LOOM applies the same idea in software: the camera AI pipeline is cut into five blocks with clear boundaries. Four of them have three quality levels (HIGH, BALANCED, ECO); one is always on.

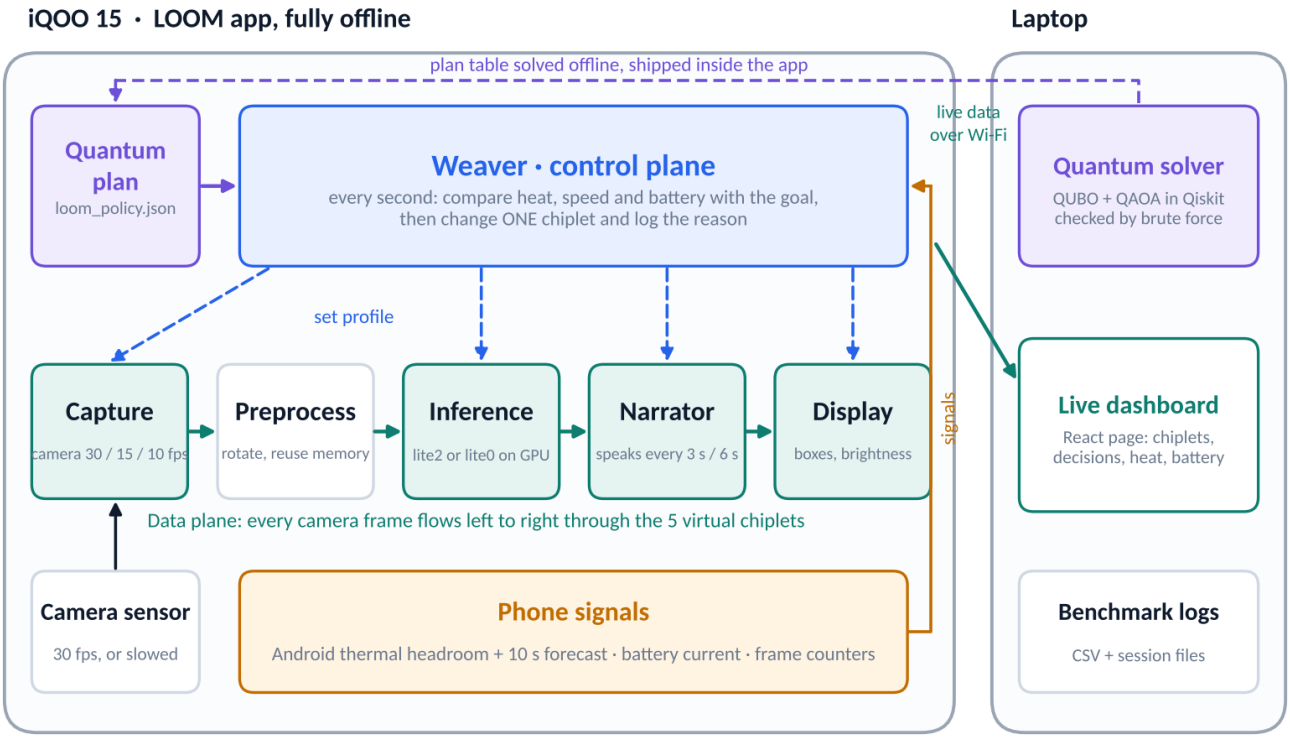


Figure 1. LOOM on the phone (left) and the laptop tools (right). Teal: the data plane, where camera frames flow. Blue: the Weaver, which sets each chiplet's level. Amber: the phone signals it reads. Violet: the quantum plan, solved offline on the laptop and shipped inside the app.

Two planes, like in a network router

- **Data plane:** every camera frame goes Capture → Preprocess → Inference → Narrator → Display. It must be fast and never stop.
- **Control plane:** the Weaver runs on its own thread, once a second. It never touches a frame; it only changes levels between frames.
- This split is why LOOM can change the AI model while the camera keeps running.

The goal the Weaver defends

> **"Keep at least 15 frames per second with the best quality possible, without letting the phone's thermal headroom go above 0.80."**
>
> If the battery is below 30% and not charging, the goal drops to 10 frames per second (battery saver). Heat always wins over speed: an overheating phone is worse than a slightly slower one.

4 · THE FIVE CHIPLETS

What each block does and how it can be tuned

Chiplet	What it does	HIGH	BALANCED	ECO
Capture	Decides how many camera frames exist and which ones are processed. Can slow the camera sensor itself.	camera 30 fps	camera 15 fps	camera 10 fps
Preprocess	Turns each frame upright (the sensor is mounted sideways) and reuses the same memory every frame. Always on.	—	—	—
Inference	Runs the object detector (a neural network) on each frame.	EfficientDet-Lite2 on GPU	EfficientDet-Lite0 on GPU	Lite0 on CPU (fallback only)
Narrator	Turns stable detections into short spoken sentences.	every 3 s	every 6 s	paused (never used for heat)
Display	Draws the boxes, smooths them, and controls screen brightness.	normal brightness	35% brightness	8% brightness

More detail on each

Capture

It first asks the **camera hardware** for fewer frames (the iQOO 15 camera offers fixed 10, 15, 24 and 30 fps modes). Frames that are never captured cost the sensor and image processor nothing. If a camera refused, LOOM would skip frames in software instead. It also **paces** processing: if the goal is 15 FPS, it processes at most 18 a second, because frames above the goal are pure heat.

Preprocess

Copies the camera image into a reused bitmap and rotates it. Reusing memory avoids creating 30 new images a second, which would make the phone's memory cleaner work hard and add heat.

Inference

Runs Google's MediaPipe object detector. The model can be **hot-swapped** while the camera runs (section 7). It returns up to 12 objects per frame; LOOM keeps only the 26 kinds that matter for walking (people, vehicles, animals, furniture, traffic lights and so on).

Narrator

Speaks only what is stable: an object must appear in at least 60% of frames over 1.5 seconds. It names at most two things, by priority (person and vehicles first), with their position (left, ahead, right) and size (near or far). It speaks only when the scene changes, with a reminder every 15 seconds. Position has a small dead zone so “left” does not flip to “ahead” and back. All speech uses the phone's offline text-to-speech.

Display

A small tracker matches boxes between frames (overlap of 30% or more) and smooths their movement, so boxes glide instead of jumping. It also controls screen brightness, which is a real power draw.

5 · WHY THESE MODELS AND SETTINGS

Every choice has a reason, and most were measured

Why MediaPipe EfficientDet-Lite

- **Two sizes with the same labels** (Lite0 and Lite2, both trained on COCO's 80 object types). Because the labels are identical, LOOM can swap one for the other mid-stream and the rest of the app does not notice.
- **Built for phones:** ready-made, float16, with a GPU path through MediaPipe's GPU delegate.
- **Open licence** (Apache 2.0), so it can ship in a product. Popular YOLO releases from Ultralytics use AGPL-3.0, which is harder to ship in a commercial app.
- **Fully offline:** the model files are inside the app. The app has no internet permission at all.

Why two model sizes (Lite2 and Lite0)

Model	Accuracy (COCO mAP)	Time per frame on the iQOO 15 GPU	Max speed	Role in LOOM
EfficientDet-Lite2	34.0	about 72 ms	about 14 FPS	Best quality, used when the phone is cool or the goal is low
EfficientDet-Lite0	25.7	about 24 ms (about 45–50 ms for the whole frame)	about 20 FPS	Default under load: fast and cooler

Lite2 alone cannot reach 15 FPS on this phone, so when the goal is 15, LOOM switches to Lite0 within seconds. With a 10 FPS goal on a cool phone, the quantum planner keeps Lite2 and slows the camera instead, because Lite2's extra accuracy is worth more there.

Why the GPU, not the CPU or the NPU

- **GPU:** fastest per frame for these models through MediaPipe, and it can be switched on and off per model.
- **CPU:** we measured Lite0 on the CPU at about 81 ms per frame, over three times slower, and it drew *more* battery (966 mA vs 783 mA at the same settings). So LOOM never chooses the CPU for heat; it remains only as a safety fallback if the GPU fails to start.
- **NPU:** another team's NPU app was criticised by the judges for heat and battery drain. MediaPipe's ready-made models do not run on the NPU directly. LOOM's point is not which chip is used but that the work is *adapted* to the heat.

Why a goal of 15 frames per second

For walking guidance, the scene changes slowly compared with a game. 15 analysed frames a second is a new look every 67 ms, more than enough to warn about a person or a chair. Every frame above that is extra heat with no benefit for the user. Under heavy heat, 10 FPS is still useful.

Why 30, 15 and 10 for the camera

The iQOO 15 camera lists fixed modes of 10, 15, 24 and 30 frames per second. LOOM uses 30 (full), 15 (half) and 10 (a third). Fixed modes give steady frame timing, which keeps detection smooth.

Why the screen is dimmed first

For a blind user the screen is the least valuable part, so it goes first. The order under heat is chosen by value to the user: **screen → model size → camera frames → how often it speaks**. Speech is never paused for heat, because for a blind user speech is the product.

Why only 26 kinds of object

A blind user walking needs to know about people, cars, bicycles, dogs, chairs, benches, steps of furniture and traffic signals, not about bottles, phones or televisions. Filtering keeps speech short and useful. (We filter after detection: asking MediaPipe to filter inside the model crashed the app on this phone, which we found and fixed during testing.)

6 · HOW THE WEAVER DECIDES

The rules, in the order they are checked

Once a second, the Weaver reads the phone's signals, compares them with the goal, and changes **at most one chiplet**. Every change is written to the decision log with its reason, for example "heat (forecast) 0.85 > 0.80 → Capture camera 30 → 15 fps".

The signals it reads

Signal	What it is	Where it comes from
Speed (FPS)	Frames actually analysed in the last second.	LOOM's own frame counter
Camera FPS	Frames the sensor produced.	A camera callback that counts every frame
Capacity	What the pipeline could do at full camera rate, from the median time per frame.	LOOM's timings
Heat	The higher of Android's thermal headroom now and its 10-second forecast (the forecast may lead by at most 0.10, so one noisy reading cannot trigger a cascade).	Android PowerManager thermal headroom
Battery	Current draw (mA), charge %, and whether it is charging.	Android BatteryManager

The decisions, checked in this order

When	What the Weaver does
Camera not delivering (app in background, screen off)	Waits. A paused camera reads 0 FPS, but a lighter profile cannot fix that, so nothing is downgraded.
Right after any change	A 5-second cooldown, so the effect of a change can be seen before the next one.
Too hot for 2 seconds (heat above 0.80)	Lowers one chiplet: first those above the quantum plan, then the reflex order screen → model → camera → speech.
Too slow for 3 seconds (more than 0.5 FPS under the goal)	Switches the detector to the smaller model. It never lowers the camera for speed, because fewer frames cannot make the app faster.
Above the quantum plan for 3 seconds	Steps toward the plan before the phone reaches the limit (feed-forward).
Comfortable for 10 seconds (fast enough and heat below 0.72)	Restores one chiplet, most valuable first, but never above the quantum plan for the current heat.

Safety details

- **Back-off:** if an upgrade has to be undone within 30 seconds, that chiplet's upgrades are paused for 30 s, then 60, 120, up to 240 s. This stops flip-flopping.
- **Hysteresis:** upgrades need heat 0.08 below the limit, and the quantum plan is looked up 0.02 hotter for upgrades, so a phone on a boundary does not bounce between two plans.
- **Battery saver:** below 30% and not charging, the goal becomes 10 FPS.
- **Floor:** if every chiplet is already at its lowest, the Weaver says so once ("every chiplet is already at its lowest profile") instead of repeating it.

7 · TWO TECHNIQUES THAT MAKE IT WORK

Changing the model without stopping, and slowing the camera itself

Hot-swap: a new AI model with no pause

Loading a neural network on the GPU takes a fraction of a second (about 0.2 to 0.8 s on the iQOO 15). If the app stopped for that long, the camera would freeze and speech would stutter. LOOM avoids that:

1. **Requested:** the Weaver asks for the new model.
2. **Built in the background:** a separate thread loads it while the old model keeps analysing every frame.
3. **Swapped at a frame boundary:** the analysis thread handles one frame at a time, so between two frames nothing is in progress. There the two models trade places: a pointer swap measured at **0.00–0.03 ms**. No frame is dropped.
4. **Old model released:** closed on the background thread, so even the 25 ms it takes to close does not delay a frame.

The dashboard's Hot-swap panel shows these four steps with their times every time it happens.

Slowing the camera sensor, not just skipping frames

Our first version skipped frames in software: the camera still captured 30 frames a second and LOOM ignored two of every three. That saves AI work, but the sensor and image processor still burned power. In real testing the phone got stuck at heat 0.84 with nothing left to change.

Now the Capture chiplet asks the camera hardware itself for 15 or 10 frames a second, on the running camera (no restart, the preview keeps going). Measured on the iQOO 15:

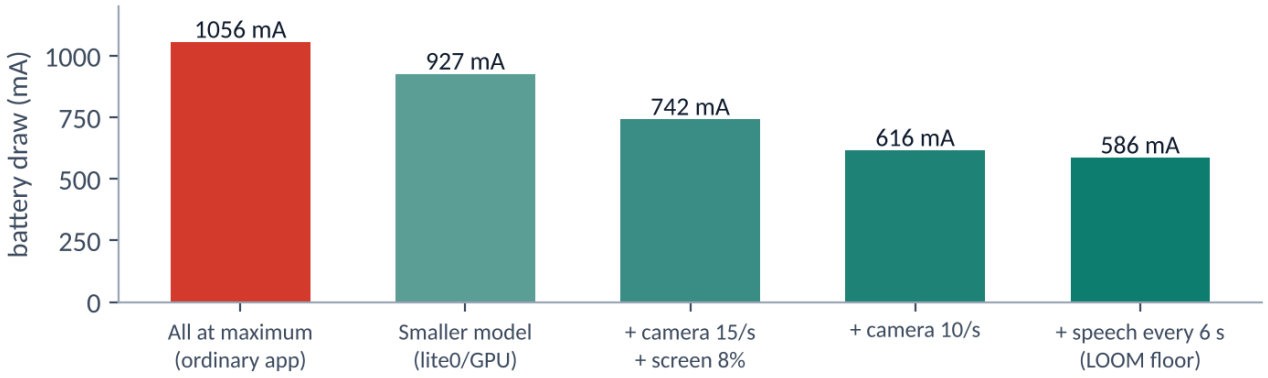


Figure 2. Battery draw for each step LOOM can take, measured on the iQOO 15 (median, unplugged). Slowing the camera from 30 to 10 frames a second saved the most.

8 · THE QUANTUM PLANNER

Choosing the best settings for every heat level, ahead of time

The Weaver reacts one step at a time. The quantum planner answers a bigger question in advance: *for this FPS goal and this heat, which level should every chiplet have, so that detection stays as good as possible while the phone stays under the heat limit for the next 3 minutes?*

Step 1 · A model built from our real measurements

- **Battery draw of every level** from the iQOO 15 runs: 1,056 mA with everything at maximum; −130 mA for the smaller model; −180 / −290 mA for camera 15 / 10 fps; −30 mA for speech every 6 s.
- **How heat responds to battery draw**, fitted to 117 one-minute windows of our runs (fit quality $R^2 = 0.82$). It predicted that at LOOM's lowest settings the heat settles at 0.837 — **the phone actually settled at 0.84**. For an ordinary app it predicted 0.975; the phone reached 0.98.
- **Quality cost of every level**, chosen by the team: screen lowest, then model size, then camera rate, then speech highest.

Step 2 · The problem as a QUBO

A QUBO (quadratic unconstrained binary optimisation) is the standard input format for quantum optimisers. LOOM's version:

- **11 qubits**, one for each (chiplet, level) pair: 3 for Display, 3 for Inference, 3 for Capture, 2 for Narrator (speech is never paused for heat). That gives 3 × 3 × 3 × 2 = **54 possible plans**.
- **Cost** = quality lost + a penalty if the detector and camera together cannot reach the FPS goal (a two-qubit term, because speed depends on the pair) + a heat price × battery draw.
- **Rules**: exactly one level per chiplet; predicted battery draw within the heat budget for the next 3 minutes. The budget becomes a price per milliamp, and a normal classical search finds the lowest price at which the plan fits (the standard Lagrangian method).

Step 3 · Solved with QAOA in Qiskit

- **QAOA** (Quantum Approximate Optimization Algorithm) prepares a quantum state in which good answers are more likely to be measured. We use 5 layers.
- **A constraint-preserving mixer** (XY mixer) moves between levels inside each chiplet without ever creating an invalid answer, so every measurement is a valid plan.
- The circuit is built gate by gate in Qiskit, checked against an exact simulation, then **measured 4,096 times** with Qiskit's sampler.

Step 4 · Every answer checked by brute force

A laptop can check all 54 plans instantly, so we do, for every one of the 15 problems (3 goals × 5 heat bands). We also build the exact version of the QUBO with extra "slack" qubits (16–17 qubits in total) and check its minimum over all 2^{16}–2^{17} bit strings. **All 15 answers match.**

> **Say it honestly**
> "It runs on a quantum simulator, not quantum hardware. With 54 plans a laptop can check them all, and we do. The point is that LOOM's decision has exactly the shape quantum optimisers solve, and it grows quickly as more chiplets and levels are added."

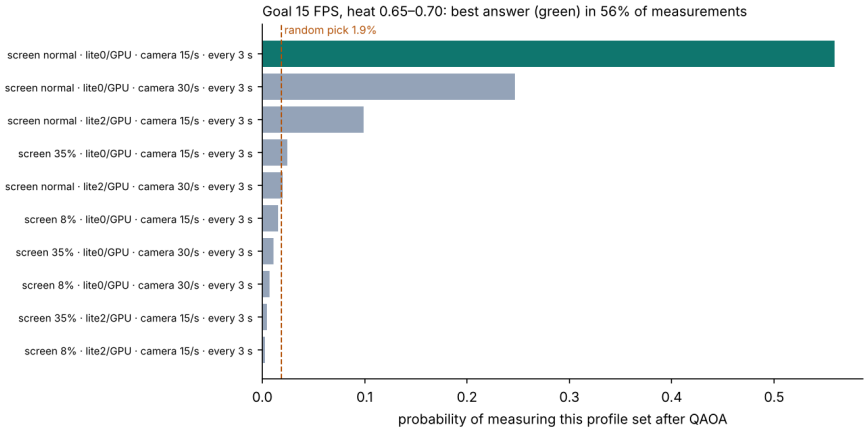


Figure 3. One of the 15 problems (goal 15 FPS, heat 0.65–0.70). After QAOA, the best plan (green) is measured 56% of the time; a random pick would be right 1.9% of the time.

The plan table, as shipped in the app

Heat band	Goal 10 FPS	Goal 15 FPS (default)	Goal 20 FPS
below 0.65	lite2 · camera 15 · screen normal · speech 3 s	lite0 · camera 30 · screen normal · speech 3 s	lite0 · camera 30 · screen normal · speech 3 s
0.65–0.70	lite2 · camera 10 · screen normal · speech 3 s	lite0 · camera 15 · screen normal · speech 3 s	lite0 · camera 15 · screen normal · speech 3 s
0.70–0.75	lite0 · camera 10 · screen normal · speech 3 s	lite0 · camera 10 · screen normal · speech 3 s	lite0 · camera 10 · screen normal · speech 3 s
0.75–0.80	lite0 · camera 10 · screen 8% · speech 6 s	same (coolest plan)	same (coolest plan)
above 0.80	lite0 · camera 10 · screen 8% · speech 6 s	same (coolest plan)	same (coolest plan)

How often QAOA measures the best plan	Number of problems
33–57% of measurements (a random pick: 1.9%)	12 of 15
10% (goal 20, heat 0.65–0.70)	1
0.6–2.4% (two problems at heat 0.70–0.75, where no single heat price makes the best plan the cheapest; it was still found among the 4,096 measurements)	2

How the phone uses it

- The table is a small file inside the app (loom_policy.json). The Weaver looks up the row for the current goal and heat every second.
- It **acts before the limit**: in testing it slowed the camera at heat 0.75 instead of waiting for 0.80.
- It **caps upgrades**: quality is never restored beyond what the plan allows for the current heat.
- The reflex rules stay underneath as a safety net if the phone is hotter than the plan expects.

9 · THE PHONE APP

What the user sees and hears



Figure 4. The launch animation: the infinity ribbon draws itself, then L and M slide in to spell L∞M (preview of the same curve and timing).

1. **App icon:** the blue infinity ribbon on black, from the LOOM logo. A plain white version is used for themed icons.
2. **Launch (about 2.6 s):** the infinity draws itself, L and M slide in, it pulses once and fades into the camera. The camera and model load underneath, so this adds no waiting.
3. **Camera screen:** only the camera view and the detection boxes. Nothing to tap: a blind user does not need buttons.
4. **Voice:** short sentences, only when the scene changes, e.g. “Person ahead, near. Chair on the left.”
5. **Engineering view (for us, not the user):** a long-press on the screen opens controls for mode, stress test, goal, heat limit and manual levels. The laptop can open it too.
6. **Offline:** the app has no internet permission. It works in airplane mode. The laptop reads diagnostics through Android's developer connection (adb) over USB or Wi-Fi.

Which numbers the phone sends to the laptop

Once a second the app writes one line of data to Android's system log (speed, heat, battery, each chiplet's level and timing, the current plan, the last decision), plus a line for every change and every hot-swap step. The laptop reads these lines live. Every frame is also written to a CSV file on the phone for later analysis.

10 · THE DASHBOARD, PANEL BY PANEL

What every number on the laptop screen means

The dashboard is a React web page served on the laptop (localhost:8765). It reads the phone's live data over Wi-Fi or USB. The screenshots below are from the built-in replay of a recorded session (it shows a blue “simulated” banner); the live view is identical.

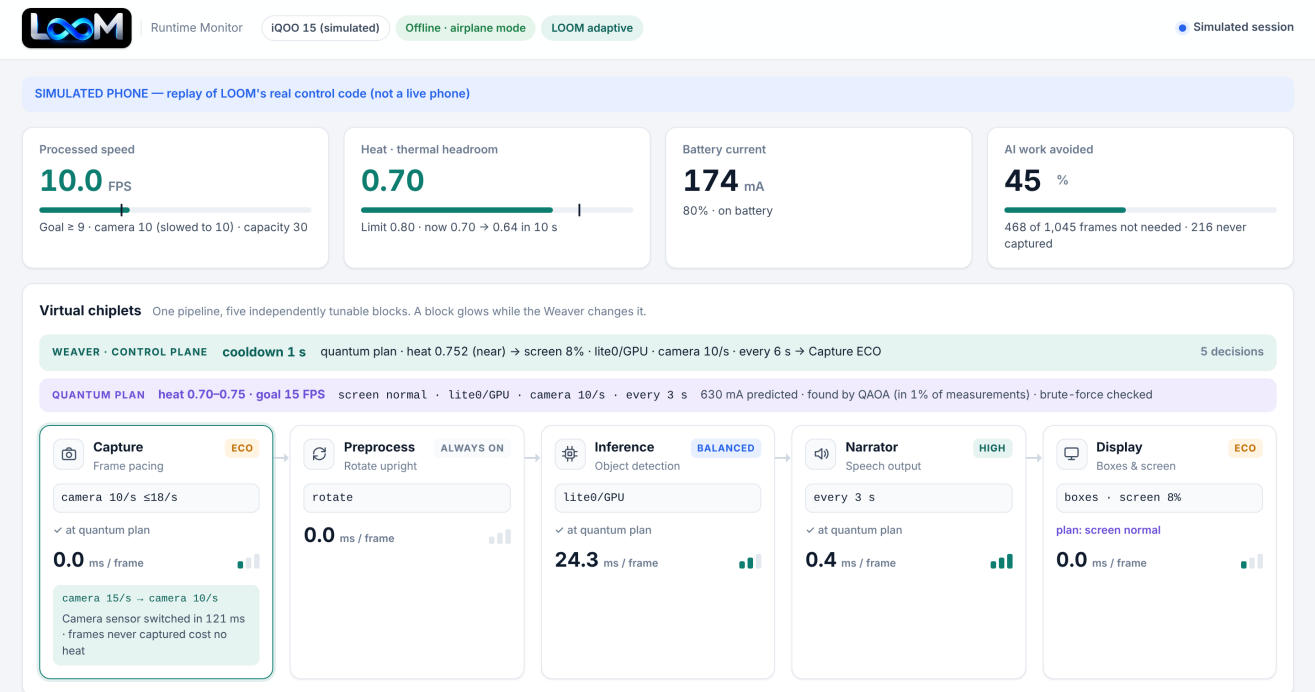


Figure 5. Top of the dashboard: header, the four live numbers, the Weaver bar, the quantum plan and the five chiplet cards.

Header

- **L∞M logo**, the phone's name, whether it is in airplane mode, and the mode: **LOOM adaptive** (Weaver in control), **Fixed** (ordinary app, for comparison) or **Manual**.
- **Top right:** “Live from the phone (Wi-Fi)” or “(USB)”, “Simulated session” in replay, or “Not connected”.

The four numbers

Card	What it shows	How to read it
Processed speed	Frames analysed per second, with the goal (the black tick), the camera's frame rate, and capacity.	Green when the goal is met. “camera 10 (slowed to 10)” means LOOM slowed the sensor.
Heat · thermal headroom	Android's heat measure (0 = cool, 1.0 = where Android throttles hard), the limit tick, and the 10-second forecast.	Red above the limit. “now 0.70 → 0.65 in 10 s” is Android's forecast.
Battery current	How many milliamps the phone draws, the charge %, and charging state.	It warns when charging, because charging adds heat and hides the real draw.
AI work avoided	Share of camera frames that did not need analysing, and how many were never captured at all.	Higher is better: work that was not needed is heat that was not made.

Weaver bar and quantum plan strip

- **Weaver bar (green):** its current status (e.g. “cooldown 3 s”, “too warm 1/2 s”, “at quantum plan”), the last decision with its reason, and how many decisions it has made.

- **Quantum plan strip (violet):** the heat band and goal it is planning for, the plan itself, the battery draw the plan predicts, and how often QAOA measured that answer. “Brute-force checked” means the answer was verified.

The five chiplet cards

- **Level badge:** HIGH, BALANCED, ECO, or ALWAYS ON for Preprocess. The three bars show the same level as a picture.
- **Profile:** the exact setting, e.g. “camera 10/s ≤18/s” (camera at 10 fps, processing paced to at most 18).
- **ms / frame:** time this block took on the last frame.
- **Plan marker:** “✓ at quantum plan”, or “plan: screen normal” if the block differs from the plan.
- **Glow and change box:** while a block is changing, its card glows and shows from → to and how it happened (“Swapped between frames · gap 0.01 ms”, or “Camera sensor switched in 121 ms”).

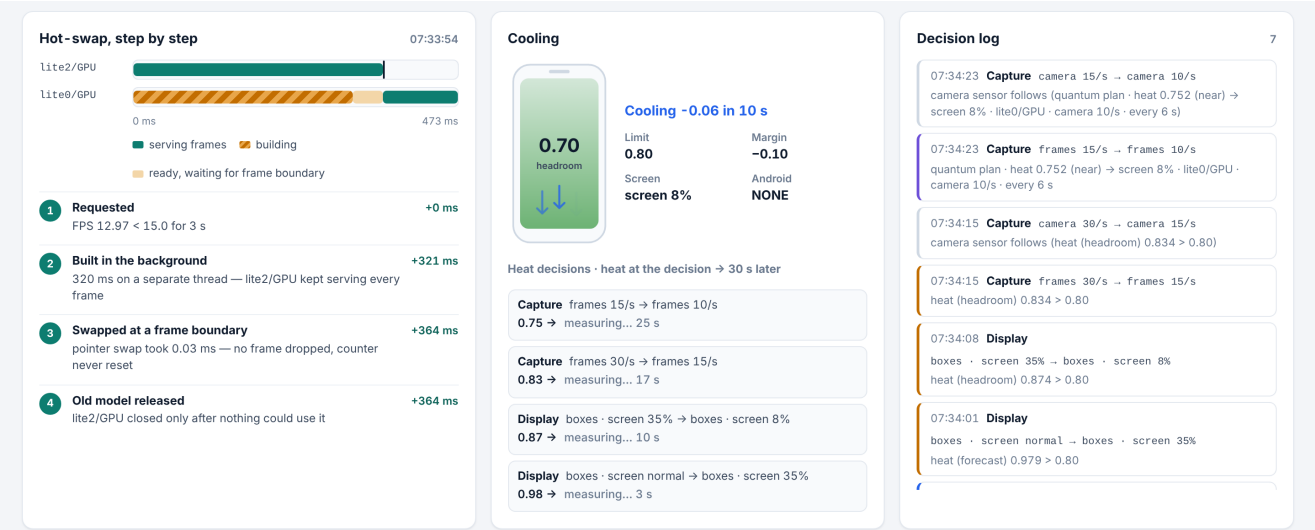


Figure 6. Middle of the dashboard: the hot-swap timeline, the cooling panel and the decision log.

Hot-swap, step by step

Two lanes show the old and the new model on a millisecond timeline: solid teal while a model serves frames, striped amber while the new one builds, light amber while it waits for a frame boundary. Below, the four steps (requested, built in the background, swapped at a frame boundary, old model released) with the time of each.

Cooling

- **The phone drawing** fills with colour from green (cool) to red (hot) with the current headroom, and arrows show whether it is warming or cooling.
- **Limit, margin, screen brightness, and Android's own thermal status** (NONE, LIGHT, MODERATE…).
- **Heat decisions · heat at the decision → 30 s later:** every heat-related change, the heat when it was made, and the heat 30 seconds later, with the difference in green (cooler) or red (hotter). This is the proof that each step worked.

Decision log

Every change, newest first, with time, chiplet, from → to, and the reason. The left edge colour tells you why: **amber** = heat, **violet** = quantum plan, **blue** = speed, **teal** = quality restored. Camera rows marked “camera sensor follows” are the hardware following a Capture decision.

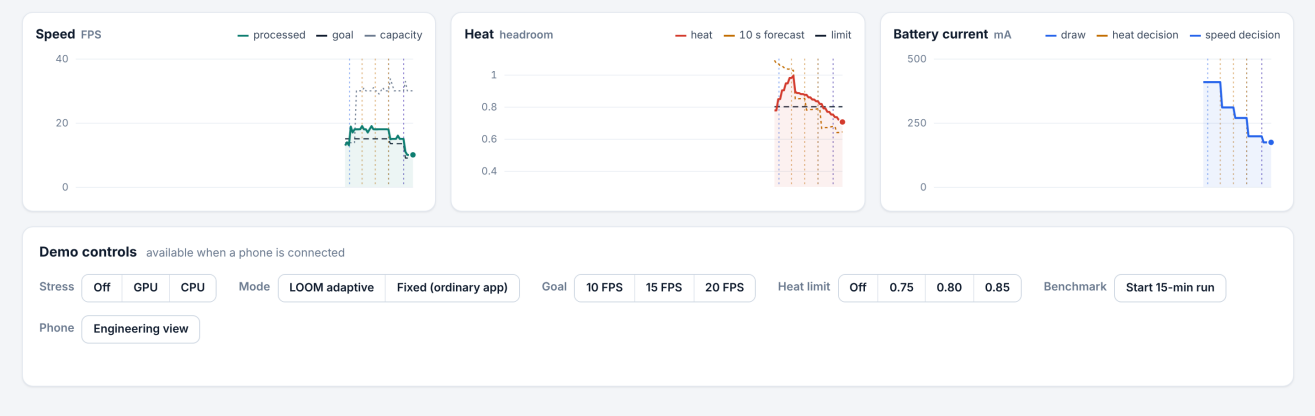


Figure 7. Bottom of the dashboard: live charts and the demo controls.

Charts

- **Speed:** frames analysed, the goal, and capacity.
- **Heat:** headroom, Android's 10-second forecast, and the limit line.
- **Battery current:** milliamps over time. Dotted vertical lines mark each decision (amber for heat, violet for the plan, blue for speed), so you can see the line step down after each one.

Demo controls (they send commands to the phone; nobody touches it)

Control	What it does
Stress: Off / GPU / CPU	Adds a hidden extra workload that heats the phone, like a heavy game in the background. Used to show LOOM reacting live.
Mode: LOOM adaptive / Fixed	Fixed = an ordinary app with everything at maximum, for comparison. Adaptive = the Weaver in control.
Goal: 10 / 15 / 20 FPS	Changes the speed goal. The quantum plan switches to the matching column.
Heat limit: Off / 0.75 / 0.80 / 0.85	Changes the heat limit. Off turns heat control and the plan off.
Start 15-min run	Starts a timed benchmark run; its data goes into its own CSV file.
Engineering view	Opens the controls overlay on the phone screen.

11 · MEASURED RESULTS

Two 15-minute runs on the iQOO 15, logged every second

Conditions: phone unplugged, airplane mode with Wi-Fi on, same room, same scene, no stress load. Run A: **Fixed** (an ordinary app: everything at maximum). Run B: **LOOM adaptive**.

	Ordinary app (Fixed)	LOOM adaptive
Heat at start → end	0.61 → 0.98, rising the whole time	0.84 → 0.85, flat
Android thermal warning	reached MODERATE	never above LIGHT
Battery draw (median)	1,048 mA	586 mA (44% less)
Speed	18.7 → 12 FPS (Android slowed the hot phone)	steady 10 FPS
Detection confidence (average)	0.55	0.53
Objects detected per frame (average)	6.4	5.9

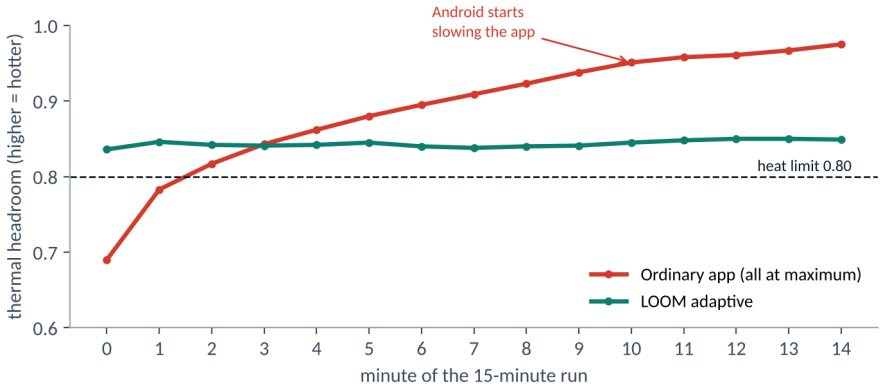


Figure 8. Heat per minute in each run. The ordinary app kept climbing until Android throttled it; LOOM held the heat flat.

How to read this fairly

- **The two runs did not start at the same heat.** The phone cooled for only 5 minutes between runs, so LOOM started at 0.84 against 0.61. At equal heat the comparison still holds: when the ordinary app reached 0.84 (minute 3) it kept climbing to 0.98, while LOOM stayed at 0.85. We plan to re-run the LOOM half from a cool phone; if that run is done before the evaluation, use its numbers.
- **LOOM stayed slightly above the 0.80 limit** because it started hot and, in that room, holding 0.80 needs about 450 mA while LOOM's lowest settings draw about 570 mA. The heat model shows this, and section 15 lists the fix.
- **One phone, one room, one day.** The numbers are real measurements, not a general promise for every phone or room.

Other measured facts worth quoting

- Hot-swap gap between models: 0.00–0.03 ms; new model built in the background in about 0.2–0.8 s.
- Camera slowed from 30 to 10 fps: battery draw fell from about 950 to 616 mA at otherwise equal settings in our logs.

12 · HOW LOOM HELPS IQOO

Software that adds to iQOO's performance and cooling story

iQOO's brand is built on sustained performance and cooling. LOOM is a software layer that works with that hardware:

1. **Sustained performance for camera AI.** An ordinary app heated up until Android slowed it from 18.7 to 12 FPS. LOOM held a steady speed with flat heat.
2. **Longer battery life with no hardware change:** 44% less draw on the same task, measured on the iQOO 15.
3. **It adds to the cooling hardware.** The vapour chamber moves heat out; LOOM stops making heat the user does not need. The two add together.
4. **Tuned per phone model.** The cost model and the quantum plan come from iQOO 15 measurements. iQOO could ship a tuned plan for each model, which a generic app cannot easily copy.
5. **Ready to become a system feature.** It uses only standard Android interfaces (thermal headroom, Camera2), so it could become a service in iQOO's OS that any camera or AI app uses. The blind-assist narrator is only our demo app.
6. **Private by design.** Fully offline: no data leaves the phone.

> **A line for the slide**
> “iQOO's hardware moves the heat out; LOOM stops making heat the user doesn't need. Measured on the iQOO 15: 44% less battery, flat temperature, no slowdown.”

Keep claims to what we measured: do not promise numbers for other iQOO models.

13 · DEMO SCRIPT FOR THE JUDGES

Five to seven minutes; let the judges press the buttons

Before they arrive

- Phone on a stand pointing at the room, **unplugged**, airplane mode on with Wi-Fi on, LOOM running.
- Laptop dashboard showing “Live from the phone (Wi-Fi)”. A second tab with the comparison chart and quantum/results/report.md.
- Backup: if Wi-Fi fails, run live_dashboard.bat --demo (it is clearly labelled as a recording).

Test	Ask the judge to...	They will see
1 · Automatic	Close LOOM and reopen it; then walk in front of the camera.	The L∞M animation, the camera with boxes, and a short sentence such as “Person ahead”. No buttons, no setup.
2 · Reacts to heat	Press Stress → GPU on the dashboard; then Stress → Off.	Within about 10 s the cards glow, decisions appear with reasons, the heat curve bends down. After Off, quality returns step by step.
3 · No-pause swap	Watch the phone screen while the Hot-swap panel updates.	The camera never freezes; the panel shows a gap of about 0.01 ms.
4 · Real savings	Pick any minute on the 15-minute chart.	Heat flat vs climbing; 586 vs 1,048 mA; the raw per-second data is in the folder.
5 · Verified quantum	Pick any row in report.md.	QAOA's answer and the brute-force answer are the same, for all 15 rows.
6 · Private (optional)	Check the phone's airplane mode.	Everything still works; no data leaves the phone.

The 3-minute talk to give while the demo runs

1. **Problem (20 s):** “Run camera AI for 15 minutes and the phone heats up, Android slows it down, the battery drains. We measured it: 0.61 to 0.98, and a third of the speed lost.”
2. **Idea (40 s):** point at the five cards. “Each block has quality levels. The Weaver checks heat, speed and battery every second and changes one block at a time, giving up the least important thing first. Models swap without stopping.”
3. **Live (60 s):** press Stress → GPU. Point at the glowing cards and the reasons in the log. “LOOM slows the camera sensor itself: frames never captured cost no heat.”
4. **Proof (40 s):** show the comparison table and chart.
5. **Quantum (40 s):** show the violet plan strip and the QAOA chart. “11 qubits, 15 problems, all match brute force. The plan lets LOOM act before the phone gets hot.”

> **Closing line**
> “Same phone, same job: 44% less battery, no overheating, and you never touched it.”

14 · QUESTIONS JUDGES MAY ASK

Short, honest answers

Is the quantum part real quantum hardware?

No, it runs on a quantum simulator in Qiskit. With 54 plans a laptop can check them all, and we do, so every answer is verified. The point is that LOOM's decision is exactly the kind of problem quantum optimisers solve, and it grows fast as chiplets and levels are added.

Did both 15-minute runs start at the same heat?

No. LOOM's run started hotter (0.84 vs 0.61). Even so, from the same heat the ordinary app kept climbing to 0.98 while LOOM stayed at 0.85. (If the cool-start re-run is done, show it instead.)

Why didn't it stay under 0.80?

In that room, staying under 0.80 needs about 450 mA and LOOM's lowest settings draw about 570 mA. Our heat model showed this. The next step is a deeper lever, such as turning the screen fully off, since a blind user does not need it.

Why not always use the lowest settings?

Detection quality would suffer when it does not need to. LOOM keeps full quality while the phone is cool and gives it up only when heat requires it.

Why the GPU and not the NPU?

We measured. The smaller model on the GPU was fastest; the CPU was three times slower and drew more battery, so LOOM never chooses it for heat. The other team's NPU app was criticised for heat; LOOM's point is adapting the work, whatever the chip.

Why only 10 FPS under heat?

That is enough for walking guidance: a new look every 100 ms. Anything above what the user needs is wasted heat.

What happens if the model swap fails?

The old model keeps running and the failure is logged. If the GPU cannot start at all, LOOM falls back to the CPU so the user still gets detection.

Does it need the internet?

No. The app has no internet permission. It works in airplane mode. The laptop only reads diagnostics through Android's developer connection.

Can it work for other apps?

Yes. The chiplet and Weaver code is separate from the narrator. Any camera or AI app with stages that have quality levels can use it; as a system service it could serve many apps.

How do you know the heat model is right?

We fitted it to 117 one-minute windows of real runs ($R^2 = 0.82$). It predicted the phone would settle at 0.837 on LOOM's lowest settings; it settled at 0.84.

Why these quality weights?

They are a design choice for a blind user: screen matters least, then model size, then camera rate, then speech. They are in one file (quantum/loom_model.json) and the plan can be re-solved in about 15 minutes with quantum_solve.bat.

15 · HONEST LIMITS AND NEXT STEPS

What we would tell a mentor in private

Limits

- The quantum planner runs on a simulator; with 54 plans, brute force is instant. QAOA demonstrates the method and is verified.
- The comparison runs started at different heat; the cool-start re-run of the LOOM half fixes that.
- In a warm room LOOM's lowest settings (about 570 mA) cannot hold 0.80 indefinitely (about 450 mA needed).
- Measured on one iQOO 15, in one room, on one day. Numbers will differ elsewhere.
- The newest build (quantum plan in the Weaver, launch animation, new icon) was compile-checked and run in a simulator before the phone test; confirm it on the phone before the evaluation.

Next steps

1. **A deeper heat lever:** let the screen go fully dark (the user is blind), and allow 5 fps camera under extreme heat.
2. **Per-model plans:** run calibrate.py on other iQOO phones and ship a plan per model.
3. **Larger plans for real quantum hardware:** more chiplets and levels (e.g. image resolution, NPU delegate) make the search space grow past what brute force handles comfortably.
4. **A system service:** expose the Weaver as an Android service so any app's AI pipeline can register its chiplets.

16 · APPENDIX

Files, scripts and a glossary

Scripts (double-click on the laptop)

Script	What it does
build_loom.bat	Builds the signed release app (LOOM.apk) and installs it on the phone over USB.
wifi_connect.bat	Switches the phone's developer connection to Wi-Fi so the cable can be unplugged.
live_dashboard.bat	Starts the dashboard. Add --demo to replay a recorded session without a phone.
pull_logs.bat	Copies the per-frame CSV logs from the phone into the csv folder.
crash_log.bat	Saves the phone's crash log to crash_log.txt if the app ever closes.
quantum_solve.bat	Re-solves the quantum plan with Qiskit (about 15 minutes) and writes the table into the app.

Main source files

Where	What
app/.../chiplets/	The five chiplets: CaptureChiplet, PreprocessChiplet, InferenceChiplet, NarratorChiplet, DisplayChiplet
app/.../weaver/Weaver.kt	The control loop and its rules; PolicyTable.kt loads the quantum plan
app/.../telemetry/	Phone signals (heat, battery), CSV logging and the live data lines
app/.../ui/	Camera screen, engineering overlay, and the L∞M launch animation
app/src/main/assets/	The two detector models and loom_policy.json (the quantum plan)
quantum/	Cost model, QUBO, QAOA solver, calibration, and results/report.md
tools/dashboard/	The React dashboard (src) and its built files (dist)
live_logs/, csv/	Recorded sessions and benchmark runs

Glossary

Term	Meaning
Chiplet (virtual)	One stage of the AI pipeline, with its own quality levels, that can be tuned on its own.
Weaver	LOOM's controller. Once a second it compares the phone's state with the goal and changes one chiplet.
Thermal headroom	Android's measure of how close the phone is to heavy throttling: around 0.6 is cool, 1.0 means Android will slow it hard.
FPS	Frames per second: how many camera images are analysed each second.
Hot-swap	Replacing the AI model while the app runs, without dropping a frame.
GPU delegate	Running the neural network on the phone's graphics processor instead of the main processor.
EfficientDet-Lite0 / Lite2	Two sizes of Google's mobile object detector; Lite2 is more accurate, Lite0 is faster.
QUBO	Quadratic Unconstrained Binary Optimisation: a problem written as yes/no variables with a cost, the input format for quantum optimisers.
QAOA	Quantum Approximate Optimization Algorithm: a quantum circuit whose measurements favour low-cost answers.
XY mixer	A QAOA ingredient that keeps exactly one level per chiplet, so every measured answer is valid.
Brute force	Checking every possible answer. With 54 plans it is instant, which is how we verify QAOA.
adb	Android Debug Bridge: the developer connection the laptop uses to read the phone's live data.